

LAB ASSIGNMENT Nº2

LoRaWAN-BASED ULTRASONIC DETECTOR

*(DRAGINO LoRa/GPS SHIELD, ARDUINO LEONARDO, THE THINGS NETWORK,
FIREBASE, ANDROID STUDIO, MQTT)*

1 INTRODUCTION

This assignment will introduce the students to Low Power Wide Area Network (LPWAN) technologies in general and LoRaWAN technology in particular. It builds upon the Lab Assignment Nº 1 in terms of persistent data storage and user interface. In fact, the Firebase project and Arduino app will require no further changes since the LoRaWAN system will seamlessly integrate with Firebase.

The instructions in this assignment are abridged so that, at each step, the students will have to investigate how to implement the requested functionality. At the end of this document, a list of useful references is provided.

2 CREATING AN ACCOUNT AND IoT APPLICATION IN THE THINGS NETWORK

This sequence steps will guide you through the creation of an account in [The Things Network \(TTN\)](#), within which an application will be registered. TTN is an open source LoRaWAN infrastructure that relies on its community of users to provide worldwide coverage. Users can register for free, create their applications and start to use the TTN infrastructure to receive data from their sensors at the TTN backend. There, the users can access the data through the Internet from any part of the World. They can also redirect that data to other servers. The users themselves may contribute installing their own LoRaWAN gateways, which are configured to connect to the router nodes and from them to the backend infrastructure. The security mechanisms of LoRaWAN allow the application data of user A to be sent to the backend through the gateway deployed by another user B, although user B is prevented from reading the data (though user B has access to the management information associated with the frames that carried the data from user A).

- 1) Go to the [TTN](#) website and create an account. If you already have an account, you can use it.
- 2) Go to the console and select the **Europe 1** (*eu1*, Dublin, Ireland) cluster. Enter the Applications area. Add a new application with the following profile:
 - a. Owner: your user ID.
 - b. Application ID: *rmic2223-group-XX*, where *XX* is the number of the group.
 - c. Application name: you can use the Application ID.
 - d. Description: *RMIC remote movement detection application*.

- 3) Once the application is created, it will be possible to add end devices. In order to be able to send/receive data to/from the backend, a device must be added. Click the option to add a device.
- 4) Select **Manually** for the type of registration of the LoRaWAN device, providing the requested parameters (see Figure 1):
 - a. Frequency plan: Europe 863-870 MHz (SF9 for RX2 – recommended).
 - b. LoRaWAN version: LoRaWAN Specification 1.0.1
 - c. Regional parameters version: PHY V1.01.
 - d. JoinEUI: 00 00 00 00 00.
 - e. Dev EUI, AppEUI, AppKey: generate these values, which will have to be configured in the Arduino program (see below).
 - f. End device ID: Some short character sequence to identify the device.
- 5) Once the device is configured, the application is ready to send/receive data to/from it. The next step is to configure and deploy the physical device.

The screenshot shows the 'Provisioning information' section of the TTN console. It includes the following fields and controls:

- JoinEUI**: A text input field with a 'Reset' button. Below it, a note states: 'This end device can be registered on the network'.
- DevEUI**: A text input field with a 'Generate' button and a '2/50 used' indicator.
- AppKey**: A text input field with a 'Generate' button.
- End device ID**: A text input field with a value 'eui-70b3d57ed0059dc7'. Below it, a note states: 'This value is automatically prefilled using the DevEUI'.
- After registration**: Two radio buttons: 'View registered end device' (selected) and 'Register another end device of this type'.
- Register end device**: A blue button at the bottom.

Figure 1: Entering device information in TTN.

3 SETTING-UP, DEPLOYING AND TESTING THE LORAWAN DEVICE AND ARDUINO CODE

The device used in this assignment will perform the same tasks as the one in Lab Assignment Nº 1, but with a different MCU and communication technology. In fact, the sensor and actuator will be the same. The MCU will now consist of an Arduino Leonardo, whose pinout is depicted in Figure 2.

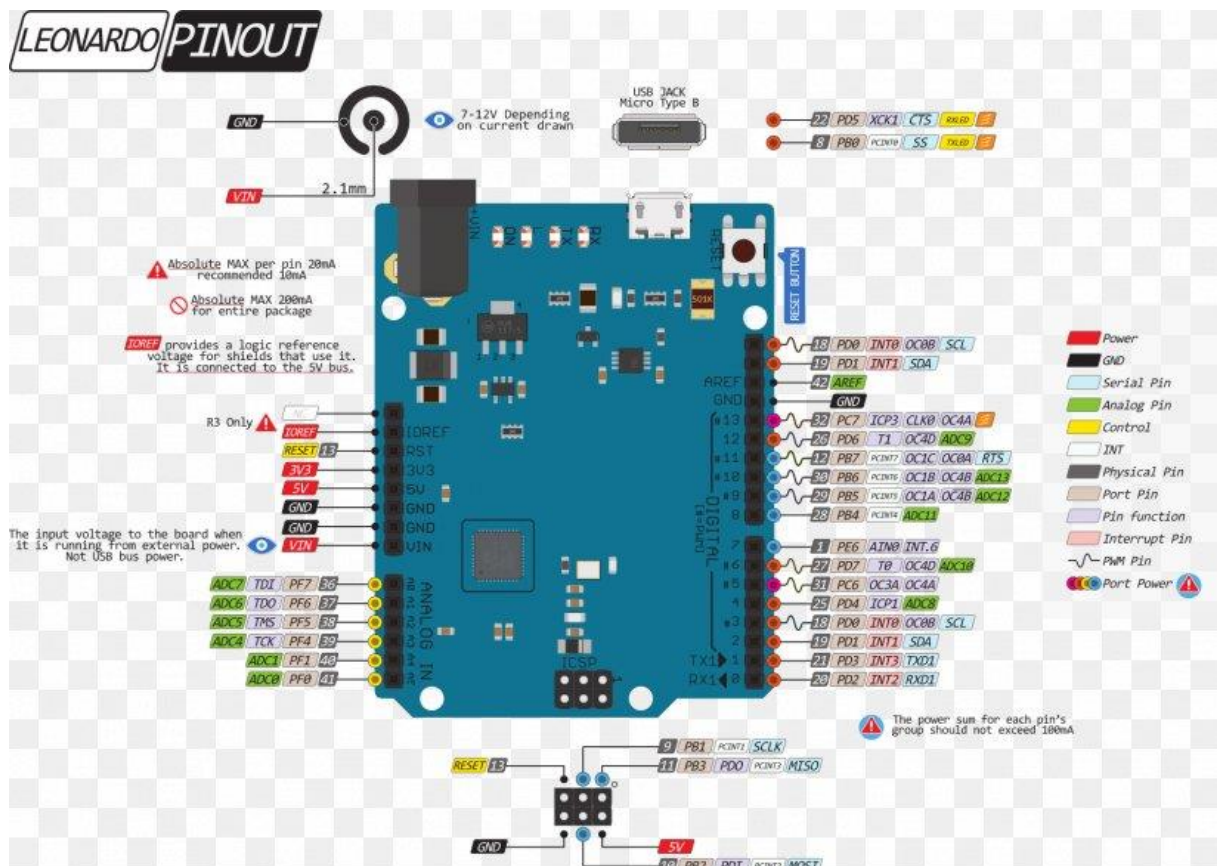


Figure 2: Pinout of Arduino Leonardo.

The LoRaWAN connectivity will be provided by the Dragino LoRa/GPS shield, whose pinout and **jumper configuration**¹ is depicted in Figure 3. The shield comes with a LoRa Bee module that implements the LoRa protocol. Although the shield implements a GPS receiver, we will only make use of the LoRaWAN functionality. Although the LoRa Bee module could be detached and used independently, the shield provides allows a more straightforward interface between the Arduino Leonardo and the LoRa Bee. In fact, most of the pins of the shield will directly connect and keep the function of the underlying Arduino pins.

¹ The jumper configuration must be carefully checked.

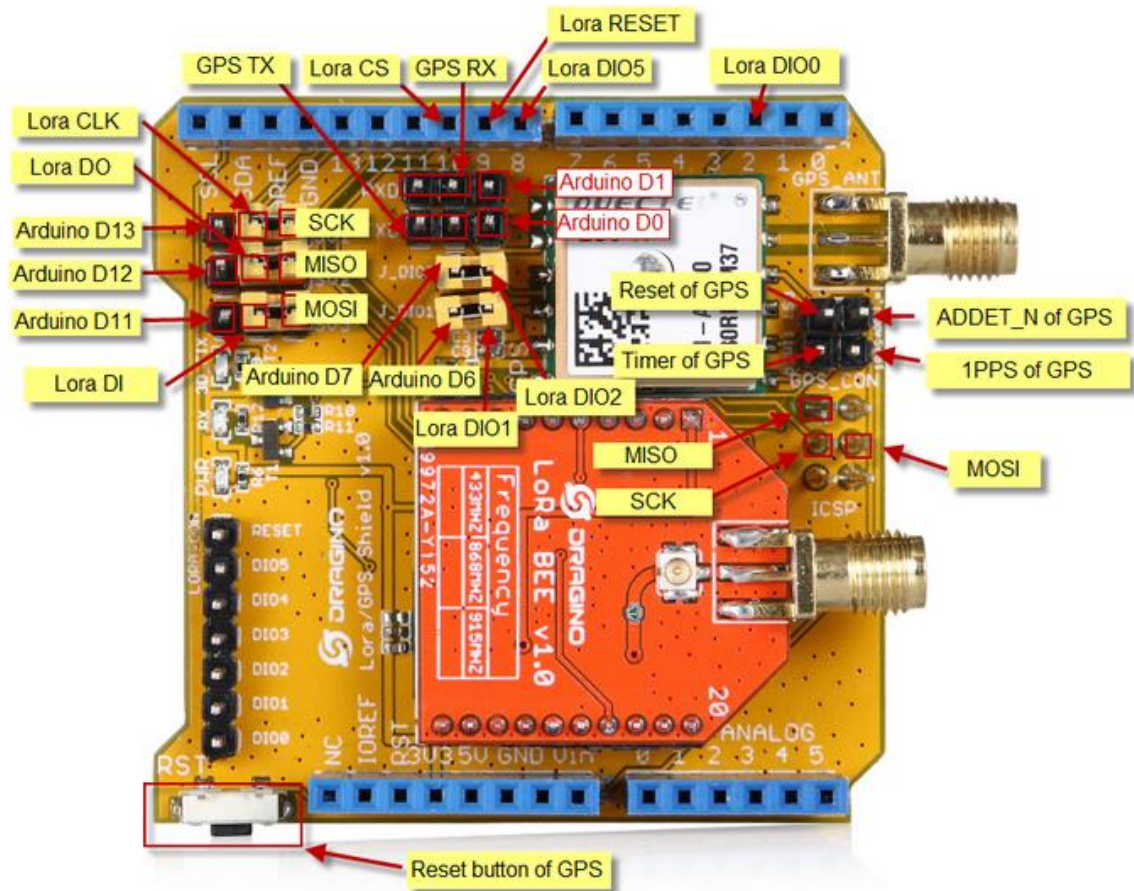


Figure 3: Dragino LoRa/GPS shield.

Arduino Leonardo will be fed by the PC through the USB interface. The LoRa/GPS shield will be attached on top of it and will connect with the HC-SR04 ultrasonic sensor, and the piezo buzzer, similarly to what was done in Lab Assignment N° 1.

- 6) Mount the circuit as depicted in Figure 4.

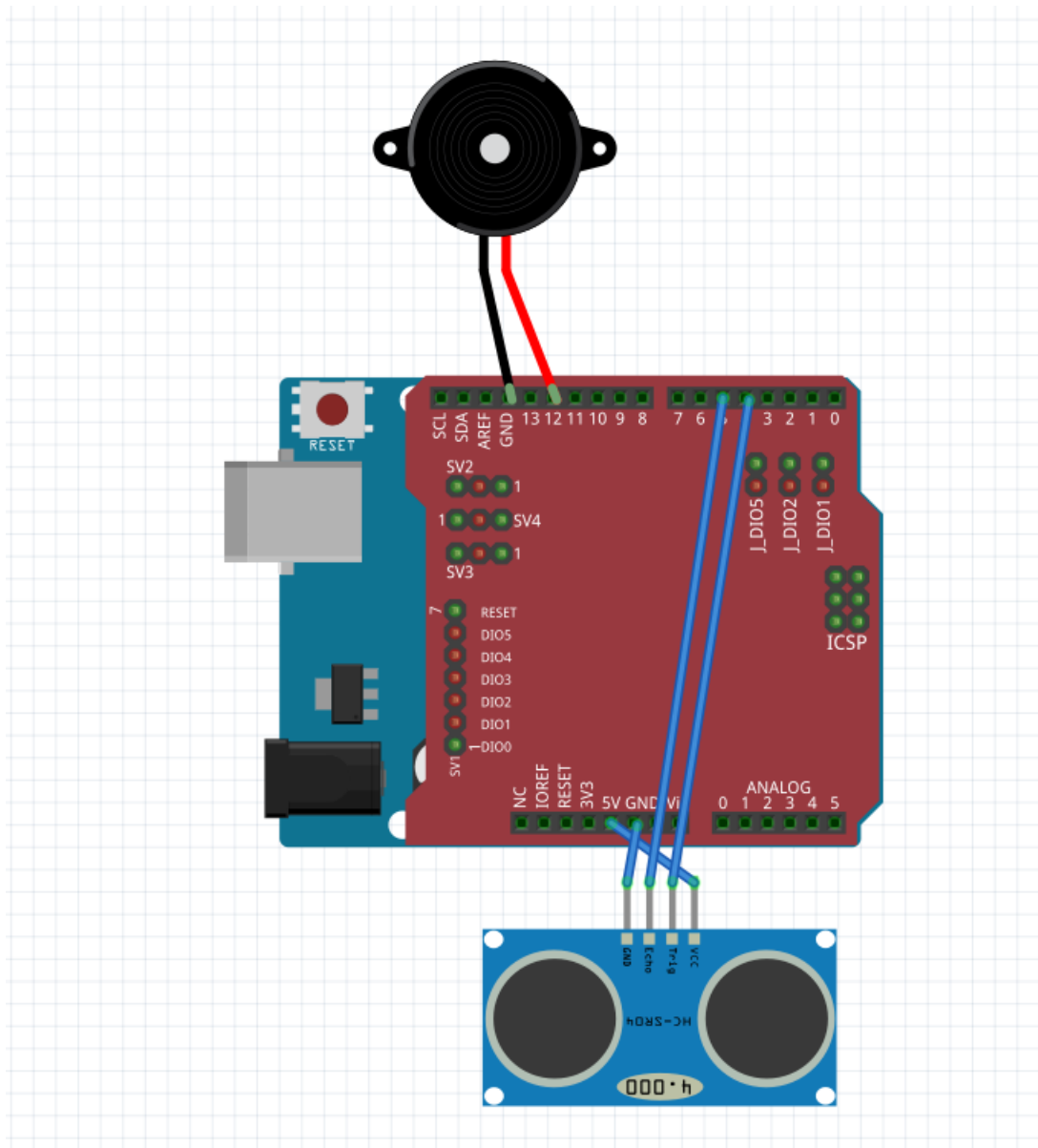


Figure 4: Ultrasonic detector and alarm circuit using Dragino GPS/LoRA shield on top of Arduino, and HC-SR04.

7) Install the IBM LMIC framework library as depicted in Figure 5.



Figure 5: Library Manager of the Arduino IDE with the IBM LMIC framework library installed.

- 8) The program that will be run in Arduino Leonardo is given in file **lorawanSRF04Sensor_v2.zip**. Configure the **Application EUI**, the **Device EUI**, and the **AppKey** with the values provided in the TTN device overview page. **Pay attention to the fact that these values must be filled in little-endian format.**
- 9) The program includes callback functions *check_sensor()* and *do_send()*, where the main actions are taken:
 - a. The *do_send()* callback function will be called periodically in order to transmit the status of the device. The format of the payload is the following (each character represents one byte): MCCB, where M stands for the status of the PIR movement detection sensor (0x01 if motion is detected, 0x00 otherwise), CC stands for the alarm count (16 bits), and B stands for the buzzer enable variable configured in downlink data (0x01 if enabled, 0x00 otherwise).
 - b. The *check_sensor()* callback function periodically checks the PIR movement detection sensor status and activates the Arduino Leonardo LED and buzzer accordingly (the latter will only be activated if the *buzzer_enable* variable has value 1). In case the sensor is currently detecting movement and the previous reading did not, the alarm count variable is incremented and the device tries to immediately transmit a packet.
 - c. The LoRaWAN technology allows the transmission of a very limited amount of data in the downlink direction, which may be used to issue commands or configuration info to the device. In this application, one byte of data can be transmitted, which will set the value of the *buzzer_enable* variable. Due to the expected energy efficiency requirements of LoRaWAN, nodes can only receive downlink data in response to uplink frames (which denote that the device is active and not sleeping). As such, expect a delay between the instant in which downlink data is scheduled and the time when it is received. The procedure to send downlink data will be addressed below.
- 10) It is useful to look at the debug messages sent to the serial interface, which can be visualized in the Serial Monitor of the Arduino IDE. Once the connectivity aspects are correct, you should be able to see the activation and data traffic in the TTN console, in the **Live data** separators of the application and of the device. You can check if the payload values are correct and match the expected device status.
- 11) The message payloads are reported in raw format. In order to read the payload data in a more user friendly way, you can set the Payload Format in the TTN application. The uplink payload format can be set in the **Payload formatters** separator of the application console. These can be set in Javascript for both uplink (device to application) and downlink (application to device). Program the payload formatters in Javascript, as shown in Figure 6 and Figure 7.
- 12) The uplink formatter defines and assigns the value of three subfields of the decoded object: *alarm_status* (Boolean), *measurement* (Integer) and *enable_alert_sound* (Boolean), whose meaning should now be familiar. Check that the decoder is well configured by looking at received uplink messages in **Live data**. The fields defined for the decoded object, as well as the respective values, should now appear reported in the logged messages.
- 13) The downlink formatter encoder allows downlink payloads to be specified and visualized in JSON format. The encoder receives a JSON object with a Boolean field named *enable_alert_sound*, transforming it into a one-byte payload of value 0x00 or 0x01,

respectively when it is *false* or *true*. The decoder assists displaying the decoded downlink message logs.

```

1 function decodeUplink(input) {
2   var bytes = input.bytes;
3   var decoded = {};
4   decoded.alarm_status = bytes[0]?true:false;
5   decoded.measurement = (bytes[1]<<8)+bytes[2];
6   decoded.enable_alert_sound = bytes[3]?true:false;
7   return {
8     data: {
9       decoded: decoded
10    },
11    warnings: [],
12    errors: []
13  };
14 }
```

Figure 6: Uplink formatter (decoder).

```

1 function encodeDownlink(input) {
2   var enable_alert_sound = ["false", "true"];
3   return {
4     bytes: [enable_alert_sound.indexOf(input.data.enable_alert_sound)],
5     fPort: 1,
6     warnings: [],
7     errors: []
8   };
9 }
10
11 function decodeDownlink(input) {
12   switch(input.fPort) {
13     case 1:
14       return {
15         data: {
16           enable_alert_sound: ["false", "true"][input.bytes[0]]
17         }
18       }
19     default:
20       throw Error("unknown FPort");
21   }
22 }
```

Figure 7: Downlink formatter (encoder and decoder).

- 14) Downlink (i.e., from the application to the device) data can be scheduled directly from the TTN console. Access the device overview page and select the **Messaging** separator. Then, under the Downlink separator, schedule downlink payloads of one byte (0x00 or 0x01) to enable or disable the buzzer. Check that the **Live data** logs appear correctly and the device behaves as expected (see Figure 8). In the **Uplink** separator, uplink messages can also be created to test the uplink formatter.

- 15) In order to avoid mistakes coding the payload formatters, file *LAB2_payload_formatters.txt* is provided, so that you can just copy/paste the code into the TTN payload formatter definitions.



↑ 19:05:39	eui-70b3d57ed0059dc7	Forward uplink data message	DevAddr: 26 0B F2 64	Payload: { decoded: { } } 00 00 00 00	FPort: 1 Data rate: SF7BW125
↓ 19:05:39	eui-70b3d57ed0059dc7	Receive downlink data message	Payload: { enable_alert_sound: "false" }	00	FPort: 1
↑ 19:05:36	eui-70b3d57ed0059dc7	Forward join-accept message	DevAddr: 26 0B F2 64		
↻ 19:05:34	eui-70b3d57ed0059dc7	Accept join-request	DevAddr: 26 0B F2 64		

Figure 8: Live data log in TTN.

4 INTEGRATING WITH FIREBASE THROUGH MQTT

By default, the TTN infrastructure will not provide persistent storage for the data. However, the uplink data can be sent to a database or to other systems if that is explicitly configured in the application by means of the Integrations mechanism. In this project, we will use an MQTT integration that will send/receive data to/from the TTN MQTT server. A Python script running anywhere in the Internet (e.g., student's laptop) will also connect to the MQTT server as a client, playing the role of gateway between MQTT and Google Firebase. In order to obtain this functionality, take the following steps:

- 1) Go to the TTN console and select the application that was already created. In the menu, select *Integrations/MQTT*. Generate a new API Key.
- 2) Prepare a new Python virtual environment directory to run the **firebase_mqtt_ttn.py** script. A virtual environment and **pip** tutorial can be found in [this webpage](#). Activate the virtual environment and then install the required packages using **pip**: *paho-mqtt*, *pyrebase4*, and *urllib3*.
- 3) Edit the *firebase_mqtt_ttn.py* script to change the Google Firebase and TTN configurations:
 - a. Change the *firebase_config* structure according to your Google Firebase account data
 - The API Key can be found in the General separator of Project Settings ⚙️;
 - The Authorized Domain can be found under the Authentication options, in the Settings separator;
 - The Database URL can be found in the Realtime Database options;
 - The Storage Bucket URL is not required and can be left as an empty string.
 - b. Change the values of APPEUI, APPID, PSW (i.e., the API Key in TTN), and DEVID according to the data provided in your TTN application webpage.
- 4) You can now run **firebase_mqtt_ttn.py** in any computer connected to the Internet.

5 REFERENCES

- [1] TTN documentation, <https://www.thethingsnetwork.org/docs/>.
- [2] The Python Tutorial, 12. Virtual Environments and Packages, <https://docs.python.org/3/tutorial/venv.html>.